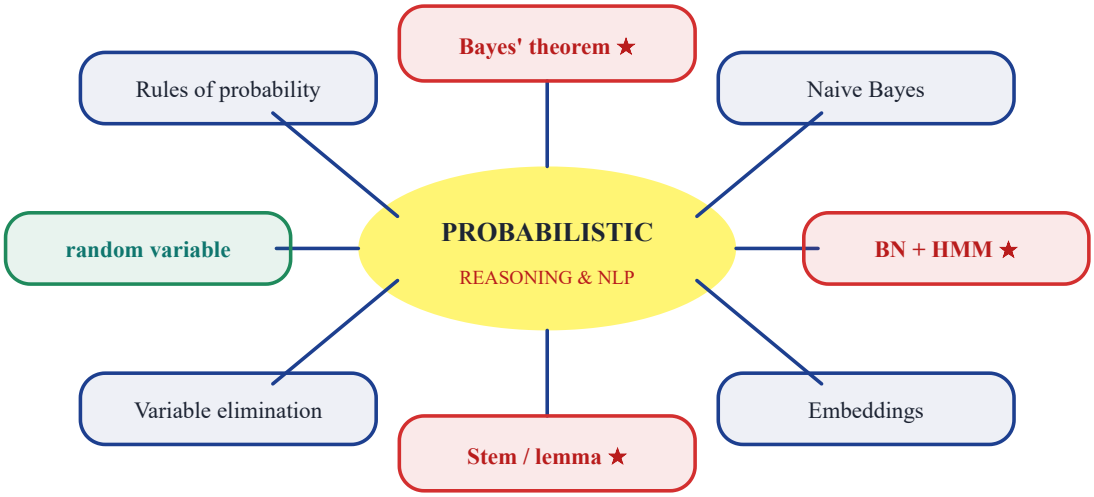
Probabilistic Reasoning & NLP

Bayes · naive Bayes · BN DAG · HMM & Viterbi · stemming · embeddings: 12 sheets



↑ the whole topic on one page: red = high-yield, green = the one idea everything hangs on

Syllabus coverage

Artificial Intelligence lists ten lines. This notebook covers exactly two: "**Probabilistic reasoning**" and "**Natural language processing**". Knowledge representation, logic, search, expert systems and machine learning have their own notes in this chapter.

High-yield list

① $P(A|B) = P(A \wedge B)/P(B)$
② $P(H|E) = P(E|H)P(H)/P(E)$
③ $P(C|x_1 \ldots x_n) \propto P(C)\prod_i P(x_i|C)$
④ BN: $\prod_i P(x_i|Pa(x_i))$
⑤ noisy-OR: $1 - \prod_i (1-p_i)$
⑥ $\alpha_t(j) = b_j(o_t)\Sigma_i a_{ij}\alpha_{t-1}(i)$
⑦ Stemming chops, lemmatisation looks up
⑧ perplexity = exp(NLL)

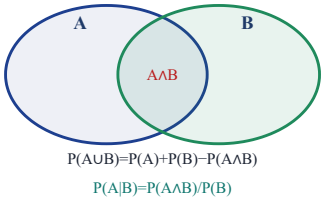
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Uncertainty & the rules of probability … p. 2
✓ Conditional probability & Bayes' theorem … p. 2
✓ Naive Bayes classification … p. 3
✓ Bayesian networks: DAG and CPT … p. 4
✓ Variable elimination, noisy-OR, D-S … p. 5
✓ Markov chains & hidden Markov models … p. 6
✓ Forward, backward & Viterbi … p. 7
✓ NLP I–III … p. 8–10 · Night-before p. 11 · Practice p. 12

1 Uncertainty, the Rules of Probability & Bayes

Definition: **Probabilistic reasoning** draws conclusions under uncertainty, using probability to say how far to believe each alternative. A **random variable** is a quantity not known until observed — **discrete** (Rain = yes/no) or **continuous** (a whole range). AI needs it because data are noisy and evidence partial.

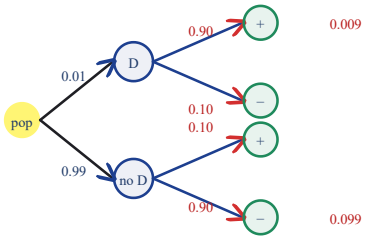


all of probability fits in two sets

Concept	Definition
Random variable	Rain = yes or no
Probability	$P(A) \in [0, 1]$
Joint	$P(A \wedge B)$, both together
Conditional	**$P(A|B)=P(A \wedge B)/P(B)$**
Marginal	$P(A) = \Sigma_B P(A \wedge B)$
Independence	$P(A \wedge B) = P(A)P(B)$

The rules you must have on your fingertips: **Axioms (Kolmogorov 1933)**: $0 \le P(A) \le 1$, $P(\Omega) = 1$, $P(A \cup B) = P(A)+P(B)$ for **disjoint** A, B. **Complement**: $P(A') = 1 - P(A)$. **Chain rule**: $P(A \wedge B) = P(A)P(B|A)$. **Total probability**: $P(B) = \Sigma_i P(B|h_i)P(h_i)$.

Bayes' theorem (1763): **$P(H|E) = P(E|H) \cdot P(H)/P(E)$**
equivalently **$P(E|H)P(H) = P(H|E)P(E)$**.
In **odds** form: **posterior odds = prior odds × likelihood ratio**. Bayes' essay appeared posthumously in 1763; Laplace (1813) gave it the first systematic use.



P(branch) = parent × edge label

Exam trap: The names are fixed: **prior** P(H) = before the evidence; **likelihood** P(E|H) = the evidence assuming H; **evidence** P(E) = the normaliser; **posterior** P(H|E) = after it.

Memory trick: "**Prior in, posterior out.**" Multiply the prior by the likelihood, then divide by the evidence; in odds form there is **no division at all**.

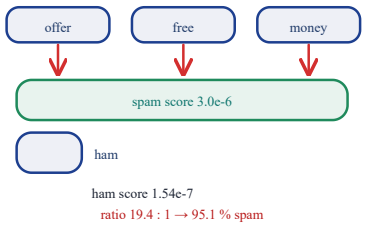
Worked example 1 ✍ the base-rate trap: A disease hits **1 %** of people; the test is **90 %** sensitive and gives **10 %** false positives. $P(D|+) = (0.9 \times 0.01)/[(0.9 \times 0.01) + (0.1 \times 0.99)] = 0.009/0.108 = \mathbf{0.083}$. A very good test still leaves a small posterior when the prior is tiny — you must always divide by the evidence.

Worked example 2 ✍ the odds form: Prior odds = 0.01/0.99 = 1/99; likelihood ratio = 0.9/0.1 = 9; posterior odds = 9/99 = 1/11, so $P = (1/11)/(1 + 1/11) = \mathbf{1/12 = 0.083}$ ✓ — the same answer with no division. Reach for odds whenever the fractions get ugly.

2 Naive Bayes Classification

Definition: A **naive Bayes classifier** picks the class C that maximises the posterior, using one conditional probability per feature. The normaliser P(x) is the same for every class, so it drops out:

$$\hat{C} = \operatorname{argmax}_C \, P(C)\cdot P(x_1|C)\cdot P(x_2|C)\cdots P(x_n|C)$$



one score per class, then compare

Piece	Meaning
$P(C)$	prior — class frequency
$P(x_i\|C)$	likelihood of x_i inside C
$\prod_i P(x_i\|C)$	the **naive step**
argmax	highest-scoring class

Three flavours: **Multinomial** (word counts): $P(w|C) = \text{count}(w,C)/\Sigma\,\text{count}(\cdot,C)$
Bernoulli (word present / absent)
Gaussian (real features): $P(x_i|C) = \mathcal{N}(x_i;\,\mu,\,\sigma^2)$

Exam trap: "Naive" does **not** mean equal priors, uniform features or linear separability — only that $\mathbf{P(x_1,\ldots,x_n|C) = \prod_i P(x_i|C)}$. Score in **log space**: thousands of small numbers underflow to zero.

Worked example ✍ scoring one mail: 100 spam and 200 ham mails. Spam holds 2 000 words (offer 40, free 60, money 30); ham holds 1 200 words (offer 5, free 10, money 8). **spam** = (1/3)(40/2000)(60/2000)(30/2000) = $\mathbf{3.0 \times 10^{-6}}$ · **ham** = (2/3)(5/1200)(10/1200)(8/1200) = $\mathbf{1.54 \times 10^{-7}}$ · ratio 19.4, so P(spam) = **95.1 %**.

Memory trick: "Multiply the **per-feature** probabilities, then add the **prior** once." Log space is what stops 10 000 features from underflowing.

The zero-frequency problem: A word never seen in class C gets P(w|C) = 0, and one zero kills the product. Fix it with **add-one smoothing**: (count+1)/(total+|V|) — 0 hits in 1 200 words with |V| = 5 000 gives **1/6200**.

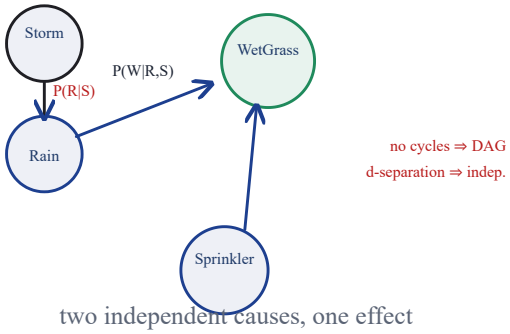
Laplace's rule of succession: After 10 heads the next is predicted with (10+1)/(10+2) = **11/12 = 0.917** — never let a count of zero become a probability of zero.

Over-smoothing, and where NB is used: Flattening the model destroys rare-word signal too. For imbalanced text use **complement naive Bayes** or **Bernoulli**. MNB is linear in the features, so it trains in one pass — which is why it survives in spam filters.

Everyday jobs: Spam filtering · sentiment analysis · named-entity recognition · news categorisation · e-mail triage · review rating · medical coding from text. "Naive" only ever means **conditional independence given the class** — never equal priors, never uniform features.

3 Bayesian Networks: DAG, CPT, Factorisation

Definition: A **Bayesian network** (belief network) is a directed acyclic graph whose nodes are random variables and whose arcs X → Y say "X is a **direct cause** of Y". Each node carries a **CPT** (conditional probability table) **P(node | its parents)**, and a parentless node holds a marginal. The whole joint is written $\mathbf{P(x_1,\ldots,x_n) = \prod_i P(x_i \mid Pa(x_i))}$ — **n** small tables instead of one with 2^n cells.



two independent causes, one effect

The graph says	Rule
Root node	stores the marginal P(X)
Child node	stores P(child | parents)
Local Markov	indep. of non-descendants **given parents**
d-separation	blocked path ⇒ independent
Why a DAG	a cycle has no topological order

CPT of WetGrass — one row per parent assignment	P(W = yes | Rain, Sprinkler on)	P(W = yes | Rain, off)	P(W = yes | no Rain, on)	P(W = yes | no Rain, off)
probability	0.90	0.80	0.40	0.10

Worked example ✍ marginalise, then condition: With P(Rain) = 0.30, P(S=on|Rain) = 0.10 and P(S=on|¬Rain) = 0.50: P(W=yes) = $\Sigma_{r,s}$ P(r)P(s|r)P(W|r,s) = 0.30[0.10(0.90)+0.90(0.80)] + 0.70[0.50(0.40)+0.50(0.10)] = 0.30(0.86) + 0.70(0.25) = **0.418**. · P(Rain|yes) = 0.243/0.418 = **0.581** — wet grass nearly triples the prior. P(Sprinkler|yes) = 0.167/0.418 = 0.399.

Memory trick: "A BN stores **one row per parent assignment**, never the whole table." The factorisation is the chain rule plus a Markov assumption at every node: **multiply the factors, then Σ over whatever you have not fixed**.

Exam trap: (i) A BN is a **DAG**: a cycle makes a variable its own ancestor, so no topological order and no valid CPT exist. (ii) Arcs mean direct dependence, **not** proven causation. (iii) Two **colliders** A → C ← B are marginally independent but become dependent once C is observed.

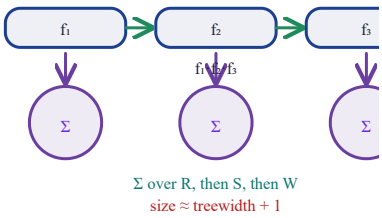
Learning the parameters: **MLE**: choose θ maximising P(data | θ) — for a Bernoulli, $\hat{p}$ = 7/20 = 0.35. **EM**: alternate an **E**-step (expected hidden labels) with an **M**-step (re-fit θ); the likelihood never decreases. EM trains mixture models and HMMs.

Three ways to answer a query: **Enumeration** — multiply the CPTs, then Σ over every hidden variable; fine up to about ten variables.
Variable elimination — the same sum, one variable at a time (next sheet).
Sampling — importance sampling or MCMC when the treewidth is hopeless.

4 Inference: Variable Elimination, noisy-OR, D-S

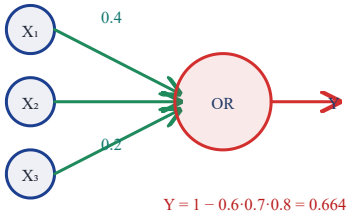
Definition: Exact inference in a BN sums over every assignment of the hidden variables — fine for three variables, hopeless for three hundred. **Variable elimination** does the same sum **one variable at a time**: multiply the factors that contain a variable, then Σ it out. The cost is the size of the largest intermediate factor, roughly the **treewidth + 1** variables, so the elimination order is the thing you actually choose.

Worked example ✍ eliminate Rain first: $f_1 = P(R)$, $f_2 = P(S|R)$, $f_3 = P(W|R,S)$. Multiply f_1 and f_2 (both contain R), then $\Sigma_R\, P(R)P(S|R) = P(S) = 0.30(0.10) + 0.70(0.50) =$ **0.38**. The factor now mentions only S, so the next product is tiny. Eliminate Rain **last** instead and you keep a factor on (R, S, W) — same answer, far more work.



multiply first, then sum out

noisy-OR — the compact cause model: For a symptom with several independent causes, any one of them is enough:
$P(Y|X_1 \ldots X_n) = 1 - \prod_i (1 - P(Y|X_i = 1, \text{others } 0))$.
0.4 and 0.3 and 0.2 give $1 - 0.6 \cdot 0.7 \cdot 0.8 =$ **0.664**, never the naive sum 0.9. It needs only **n** parameters instead of 2^n.



any one cause is enough to fire Y

Dempster–Shafer — evidence that admits ignorance: Mass m(A) is belief assigned to a **focal element**, a set rather than a point. Under the **closed-world** assumption **$m(\emptyset) = 0$**, and every probability is bracketed: **$Bel(A) = \Sigma_{B \subseteq A}\, m(B) \le P(A) \le \Sigma_{B \cap A \neq \emptyset}\, m(B) = Pl(A)$**. Dempster's rule combines two sources and normalises by the conflict $k = \Sigma_{B \cap C = \emptyset}\, m_1(B) m_2(C)$.

Exam trap: Bayes assumes **one true value** whose probabilities sum to 1; D-S admits **ignorance**, so it can only **bracket** a probability. Bel ≤ Pl, never the reverse. Two sensors both giving 0.6 on {s} and 0.4 on {s, f} combine to Bel(s) = **0.84** and Pl(s) = **1.00**.

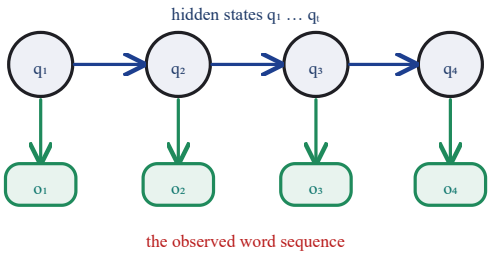
Memory trick: Three ways to combine evidence, three shapes: **Σ** = sum a variable out (elimination), **∏(1−p)** = noisy-OR, **$m(\emptyset) = 0$ plus a bracket** = Dempster–Shafer. In every case: **multiply first, normalise afterwards**.

IT link: Variable elimination is literally a join order in a database: a chain of **β-joins**. Multiplying factors = joining relations; summing a variable = projecting a column away. "Smallest scope first" is exactly the rule a query planner uses.

5 Markov Chains & Hidden Markov Models

Definition: A **Markov chain** is a discrete-time process in which the next state depends on the present state alone: $\mathbf{P(X_{t+1}|X_t,\ldots,X_0) = P(X_{t+1}|X_t)}$. The probabilities sit in a **transition matrix A** whose **rows all sum to 1**; after k steps the distribution is πA^k. An **HMM** adds an unobservable state — you see only noisy **observations** emitted by the states.

The HMM 5-tuple	What it is
$S = \{s_1 \ldots s_N\}$	hidden states
$O = \{o_1 \ldots o_M\}$	observation symbols
$A = [a_{ij}]$	transition matrix
$B = [b_j(o)]$	emission probs
π	initial distribution



the states march on; only emissions are seen

Weather	Sun	Cloud	Rain
Sun	0.5	0.3	0.2
Cloud	0.2	0.4	0.4
Rain	0.4	0.4	0.2

Three classical problems: **Evaluation** — how likely is this string? P(O) = Σ over all paths.
Decoding — which hidden path explains it? argmax.
Learning — fit A, B, π from data: **Baum–Welch** (EM).

Where HMMs earn their keep: Speech recognition · POS tagging · spelling correction · gene finding (DNA is a four-letter observation alphabet) · handwriting · anomaly detection. The future depends only on the present state, the observation only on the current one.

Worked example ✍ a two-step weather chain: Day 1: (0.5, 0.3, 0.2). Day 2 = πA = (0.39, 0.35, **0.26**). Day 3 = (0.369, 0.361, **0.27**). The rows always sum to 1 — the quickest check that your arithmetic is still alive.

Memory trick: "**Markov = memoryless. Hidden Markov = the states are hidden, the observations are visible.**" Rows of A sum to 1; the 5-tuple is **S, O, A, B, π**; a symmetric k-state chain has stationary distribution 1/k.

Exam trap: A plain Markov chain is **fully observed** — there is nothing to decode. The **MDP** (states, actions, transition probabilities, rewards) and reinforcement learning belong to the expert-system and machine-learning notes.

Common misconception: "Hidden" does not mean hidden from the algorithm — it means hidden from the **observer**. The algorithms on the next sheet recover the most likely path from exactly the same five quantities. PageRank is a Markov chain over web pages with damping d = 0.85.

6 Forward, Backward & Viterbi Decoding

Forward — sum every path: $\alpha_t(j) = P(o_1 \dots o_t, q_t{=}s_j) = \mathbf{b_j(o_t)\cdot\Sigma_i\, a_{ij}\, \alpha_{t-1}(i)}$
$\alpha_1(j) = \pi_j b_j(o_1)$ · $P(O) = \Sigma_j\, \alpha_T(j)$ · cost $O(TN^2)$.

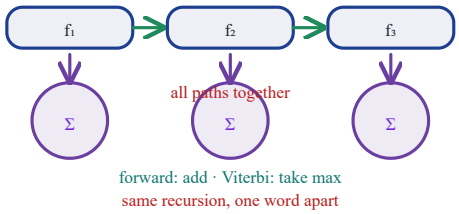
Backward — what is still possible: $\beta_t(j) = P(o_{t+1} \dots o_T \mid q_t{=}s_j) = \mathbf{\Sigma_i\, a_{ji}\, b_i(o_{t+1})\, \beta_{t+1}(i)}$
$\beta_T(j) = 1$ · smoothing: $P(q_t{=}s_j|O) = \mathbf{\alpha_t(j)\beta_t(j)/P(O)}$

Viterbi — the single best path: $\delta_t(j) = \max_i [\delta_{t-1}(i)\cdot a_{ij}] \cdot b_j(o_t)$, with ψ storing the argmax, then backtrack along ψ. The whole difference from forward: **max, not Σ**.

Exam trap: Forward gives **P(O)**, Viterbi gives the **best path**, and the best path is **not** the most probable state at every instant. Long sequences underflow: divide by $c_t = \Sigma_j\, \alpha_t(j)$ each step and multiply it back at the end.

Worked example 1 ✍ three states, observations (dry, dry, wet): π = (0.5, 0.3, 0.2); A as on the previous sheet; b(dry) = (0.9, 0.6, 0.1), b(wet) = (0.1, 0.4, 0.9). · α_1 = (0.45, 0.18, 0.02) · α_2 = (0.1998, 0.1926, 0.0107) · α_3 = (0.0122, 0.0798, 0.0738) · **P(O) = 0.1658** · δ_3 = (0.0065, 0.0324, 0.0365) ⇒ best path $\mathbf{S_1 \to S_2 \to S_3}$.

Worked example 2 ✍ marginals differ from the best path:
$\gamma_3 = \alpha_3(j)\beta_3(j)/P(O)$ = (0.073, 0.481, 0.445): the most likely state at t = 3 is S_2, yet the most likely **path** ends in S_3. Marginal ≠ path.



forward sums over paths; Viterbi maximises

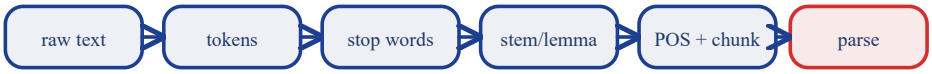
Algorithm	Question it answers	Combines with
Forward	P(O) — how surprised are we?	Σ over all states
Backward	the "future" weight of state j	Σ over all states
Forward–backward	$P(state_t\|O)$ — **posterior smoothing**	α(j)β(j)/P(O)
Viterbi	the best single path	max over states
Baum–Welch	learns A, B and π (EM)	E-step, M-step

Memory trick: **"Forward adds, Viterbi takes the max, backward looks back."** Write forward once; then **max** replaces the Σ, ψ replaces the α, and you have Viterbi.

IT link: These recursions are a **dynamic-programming shortest path over a lattice** — the same shape as shortest path in a weighted DAG, and the same shape as the beam search inside a speech recogniser.

7 NLP I: Pipeline, Morphology, Tokenisation

Definition: **Natural language processing** (NLP) is the branch of AI that lets a computer **understand, interpret, generate and respond** to human language, closing the gap between human communication and computer understanding. It is a **pipeline**: every stage consumes the tokens the previous stage produced, so a mistake at the bottom is never repairable at the top.



morphological → lexical → syntactic → semantic → pragmatic → discourse

every arrow is a place where a wrong decision becomes permanent

Levels of analysis: **Morphological** — words are built from a root plus affixes.
Lexical — split into tokens, look each one up.
Syntactic — POS tags, chunks, parse trees.
Semantic — meaning of words and sentences.
Pragmatic / discourse — intent, sarcasm, cohesion.

Morphology ✍️: A **lexeme** is one word-form family — go, goes, went, gone are four forms of **one** lexeme. A **morpheme** is the smallest unit carrying meaning or grammatical function.

Affixes: The **root** carries the core meaning; affixes modify it — **prefix** (un-), **suffix** (-ing), **infix** (un**bel**ievable), **circumfix** (German *ge…t*).

Stop words: Frequent function words carrying little content — *the, of, and, a, is* in English, the equivalents in Bangla. Removing them shrinks the index, but **they carry negation and emphasis** — "not good" is not "good".

Memory trick: "Morphology builds words, tokenisation cuts them, stop words go out, stems come back." Six levels in one order: **M L S S P D** = morphological, lexical, syntactic, semantic, pragmatic, discourse.

IT link: The pipeline is a real ETL: tokenisation is the **split**, stop-word removal the **filter**, stemming the **projection**, and the chain ends in an **inverted index** — the structure behind every search engine.

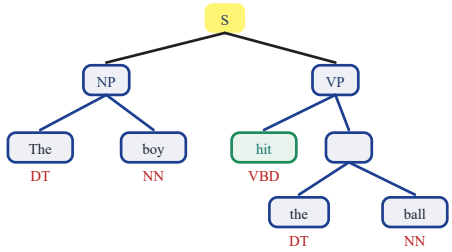
Tokenisation ✍️: Splitting raw text into **tokens**: word-level, sentence-level, and **subword** — **BPE**, **WordPiece**, **Unigram LM** — which cut rare words into seen pieces. "The Bank of Dhaka didn't approve my loan!" gives ten tokens.

Why it is hard ✍️: Bangla has no clear word boundary and drops most case endings, so whitespace tokenisation fails. Sentence segmentation is the same problem one level up: the full stop after "Ltd." does not end a sentence.

8 NLP II: Stemming, POS, Chunking, Parsing

Stemming vs lemmatisation: **Stemming** chops suffixes with heuristic rules (Porter, **1980**): relational → relat, connected → connect — the output is **not a real word**. **Lemmatisation** uses a vocabulary plus the part of speech: better → good, teeth → tooth.

Word	Stem	Lemma	Tag
playing	play	play	VBG
played	play	play	VBD
better	better	**good**	JJR
flies	fli	**fly**	NNS
mice	mice	**mouse**	NNS



constituency parse, POS tag under each leaf

POS tagging & chunking: **POS tagging** labels every token's part of speech. **Penn Treebank: 45 tags** (a 36-tag reduced set is also quoted); the **Universal POS** set has **17 tags**. **Chunking** (shallow parsing) groups tags into flat phrases — NP, VP, PP.

Worked example 1 ✍ tag the sentence: "The *flies* hit the wall" → The/DT flies/NNS hit/VBD the/DT wall/NN. But "The flies *hit* the wall" is a complete grammatical sentence too, and only context decides.

Parsing: **Constituency parsing** uses a context-free grammar and produces nested phrases like the tree on the right. **Dependency parsing** links every word to its grammatical **head**; Universal Dependencies defines **16 universal relations** — nsubj, obj, det, amod, ROOT, case, nmod.

Memory trick: "Stemming is the **scissors**, lemmatisation is the **dictionary**." A POS tag says what kind of word it is, chunking adds one level of phrase, parsing adds all of them. Counts: **45** Penn tags · **17** Universal POS · **16** UD relations.

Exam trap: **POS tagging is not parsing** — a tagger answers "what kind of word is this?", a parser "how do the words hang together?". Stemming is **not** lemmatisation. And "flies" alone carries **three** readings: plural noun, third-person verb, or acrobats.

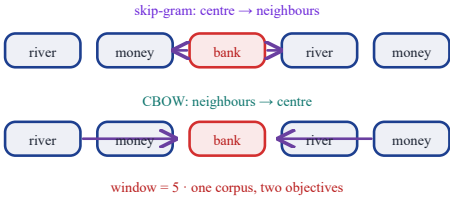
Ambiguity — four types, all tested: **Lexical** (bank: river / money) · **Syntactic** ("I saw the man with the telescope") · **Semantic** ("He kicked the bucket") · **Pragmatic / scope** ("He saw her with the dog").

Worked example 2 ✍ chunk the sentence: "The boy hit the ball in the park" → [**NP** The boy] [**VP** hit the ball] [**PP** in the park]. Chunking stops there — it never says which noun phrase attaches to which verb.

IT link: An **inverted index** stores, for every term, the sorted list of documents containing it; chunks and dependency arcs sit beside it. A ranker then scores each (document, term) pair — the same shape as a naive Bayes document score with TF-IDF weights.

9 NLP III: Embeddings & n-gram Language Models

Definition: A **word embedding** maps every word to a short dense vector so that words used in similar contexts end up near each other — Firth's distributional hypothesis, *"you shall know a word by the company it keeps"*. One-hot vectors are the opposite: sparse, all distances equal, no semantics.



two objectives, one set of vectors

Model	Idea
Word2Vec (2013)	CBOW from context; skip-gram from centre word
GloVe (2014)	factorises **global** co-occurrence counts
FastText (2016)	embeds **subwords**, so rare words get vectors

n-gram language model: The chain rule $P(w_1 \ldots w_T) = \prod_t P(w_t|w_1 \ldots w_{t-1})$ is exact but needs the whole history. Markov's approximation keeps only n−1 words: $\mathbf{P(w_t|w_{t-2:t-1})}$ is a **bigram**, $\mathbf{P(w_t|w_{t-3:t-1})}$ a **trigram**. Longer n-gram = better model, far more sparsity.

Exam trap: The **zero-frequency problem** is the whole reason smoothing exists. **Add-one**: $P(w_i|w_{i-1}) = (c+1)/(c_{\cdot}+|V|)$. **Interpolation** blends unigram and bigram. **Kneser–Ney** uses **continuation** counts. **Perplexity** = exp(per-token NLL): lower is better.

Worked example 1 ✍ a bigram: N = 19 words, |V| = 6 (+1 for <unk>). P(I) = 5/19; P(am|I) = 2/5 = 0.4; P(a|am) = 2/2 = **1.0**; P(student|a) = 3/4 = 0.75. So P("I am a student") = 0.2632 × 0.4 × 1 × 0.75 = **0.0789**. That 1.0, built from two observations, is overfitting in miniature.

Worked example 2 ✍ add-one smoothing: With |V| = 7: P(I) = 6/26 = 0.2308, P(am|I) = 3/12 = 0.25, P(a|am) = 3/9 = **0.333**, P(student|a) = 4/11 = 0.3636, so the same sentence drops to **0.0070**. Smoothing costs accuracy on frequent words and buys you no zeros.

Worked example 3 ✍ perplexity picks the model: Two models predict three tokens with probabilities (0.4, 0.4, 0.2) or (0.2, 0.2, 0.6). Mean NLL is **1.147** against **1.243**, so the perplexities are e^1.147 = **3.15** and e^1.243 = **3.47**. The first model wins — **lower perplexity is better**.

Memory trick: "**Longer n-gram, better model, more zeros.**" Embeddings answer "which words are alike?" as **points in space**; language models answer "what comes next?" as a **probability**. **king − man + woman ≈ queen** is a property of the geometry, not a programmed rule.

Exam trap + timeline: A word embedding is **not** a language model, and Word2Vec is **not** trained from labels. Four eras of NLP: **rule-based → statistical** (HMM, n-grams) → **machine learning → deep learning** (RNN, LSTM, then the Transformer of 2017, BERT 2018, GPT).

Night-Before-the-Exam Sheet

Formula sheet:

$P(A|B) = P(A \wedge B)/P(B)$

$P(A \cup B) = P(A) + P(B) - P(A \wedge B)$

$P(A \wedge B) = P(A)P(B|A)$ — chain rule

$P(B) = \Sigma_i P(B|h_i)P(h_i)$

$P(H|E) = P(E|H)P(H)/P(E)$

post. odds = prior odds × LR

$P(C|x_1 \ldots x_n) \propto P(C)\prod_i P(x_i|C)$

smooth: $(c+1)/(c.+|V|)$

$P(x_1 \ldots x_n) = \prod_i P(x_i|Pa(x_i))$

noisy-OR: $1 - \prod_i(1-p_i)$

$Bel(A) \leq P(A) \leq Pl(A)$

ppl = exp(NLL)

$\alpha_t(j) = b_j(o_t)\Sigma_i a_{ij}\alpha_{t-1}(i)$

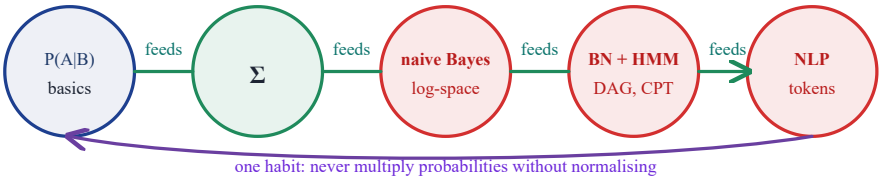
$\gamma_t(j) = \alpha_t\beta_t/P(O)$

Top 12 one-liners for MCQs

1. Kolmogorov's three axioms: **1933**
2. Bayes' theorem **1763**; Laplace's use **1813**
3. Naive = independence **given the class**
4. Add-one: **(c+1)/(c.+|V|)**
5. BN = **DAG**, one CPT row per parent set
6. noisy-OR: 0.4, 0.3, 0.2 → **0.664**, not 0.9
7. Markov = memoryless; HMM = **S, O, A, B, π**
8. Forward sums, Viterbi takes the **max**
9. Best path ≠ most likely state each step
10. Penn **45** · Universal POS **17** · UD **16**
11. Porter stemmer **1980**; stop words kill negation
12. Perplexity = **exp**(NLL) — lower is better

Top traps: Belief function is **not** likelihood and posterior is **not** prior · "naive" is conditional independence, not equal priors · a BN is a **DAG**, never a cyclic graph · noisy-OR is **1 − ∏(1−p)**, not a sum · hidden means hidden from the **observer**, not from the algorithm · Viterbi maximises one path while forward sums them all · stemming ≠ lemmatisation · POS tagging ≠ parsing · an embedding is not a language model · perplexity is an exponential, not a probability.

All tricks together: "Prior in, posterior out" (Bayes) · "Multiply per-feature probabilities, then add the prior once; work in log space" (naive Bayes) · "One row per parent assignment" (BN) · "Multiply first, sum out after; smallest scope first" (elimination, noisy-OR, D-S) · "Markov = memoryless; hidden = hidden from the observer" (HMM) · "Stemming is the scissors, lemmatisation is the dictionary" (NLP) · "Longer n-gram, better model, more zeros" (language models).



four ideas in, one habit out — quantify the doubt instead of hiding it

✓ Practice MCQs (exam pattern, 10 marks)

1. $P(A|B) = 0.4$, $P(A) = 0.3 \Rightarrow$:
(a) 0.7 (b) $P(A \wedge B) = 0.12$
(c) 1.33 (d) 0.24

2. Prevalence 1 %, test 99 % sensitive, 99 % specific:
(a) 0.99 (b) 0.10
(c) ≈ 0.50 (d) 0.01

3. "Naive" in naive Bayes means:
(a) indep. given class (b) equal priors
(c) linear separ. (d) uniform features

4. A Bayesian network factorises:
(a) $\prod_i P(x_i)$ (b) max over all
(c) $\Sigma_i P(x_i \mid Pa)$ (d) $\prod_i P(x_i \mid Pa(x_i))$

5. A Bayesian network must be a DAG:
(a) arrows point down (b) cycle = own ancestor
(c) CPT needs 2 parents (d) joint is uniform

6. noisy-OR, two causes of 0.4:
(a) 0.80 (b) 0.16
(c) 0.64 (d) 0.36

7. True of Dempster–Shafer:
(a) $Bel \le P \le Pl$ (b) $Pl \le Bel$ always
(c) one prob per event (d) = naive Bayes

8. Forward and Viterbi differ:
(a) forward ignores data (b) Viterbi is exact
(c) different HMMs (d) sum vs max one path

9. "flies" → "fly" with a dictionary:
(a) stemming (b) lemmatisation
(c) tokenisation (d) chunking

10. True of word embeddings:
(a) one-hot = semantics (b) GloVe = parallel pairs
(c) king & queen near (d) trigram = embedding

Answer key → 1-b · 2-c · 3-a · 4-d · 5-b · 6-c · 7-a · 8-d · 9-b · 10-c

Written question: State Bayes' theorem with its four terms. A test detects a disease in 90 % of cases, the disease affects 1 % of the population and the test is positive for 10 % of healthy people — find P(disease | positive), then write the naive Bayes rule and say why it is called naive. 10 marks

1. **Theorem (3)** — $P(H|E) = P(E|H)P(H)/P(E)$. Prior $P(H)$: before the evidence. Likelihood $P(E|H)$: the evidence if H is true. Evidence $P(E)$: the normaliser. Posterior $P(H|E)$: after. 3
2. Numerator $0.9 \times 0.01 = 0.009$; denominator $0.009 + 0.1 \times 0.99 = 0.108$. 2
3. $P(D|+) = 0.009/0.108 =$ **0.083** — a good test, a tiny prior, a small posterior. 2
4. **Rule (2)** — $\hat{C} = \text{argmax}_C\, P(C)P(x_1|C)\cdots P(x_n|C)$; the normaliser $P(x)$ cancels. 2
5. **Why naive (1)** — it assumes $P(x_1,\ldots,x_n|C) = \prod_i P(x_i|C)$: features independent given the class. 1

Before the exam ✓

- ✓ $P(A|B)$, chain rule
- ✓ Bayes in odds form
- ✓ score a document
- ✓ sketch a BN, enumerate
- ✓ forward, backward, Viterbi
- ✓ name 45 / 17 / 16